NOTE Hosting an ASP.NET Core app as a Windows Service can be useful if you don't want to (or can't) use Internet Information Services (IIS). Some older versions of IIS don't support gRPC, for example. By hosting as a Windows Service, your application can be restarted automatically if it crashes.

You can follow a similar process to install a worker service as a system daemon by installing the Microsoft.Extensions.Hosting.Systemd package and calling `UseSystemd()` on your `IHostBuilder`. For more details on configuring system, see the "Monitor the app" section of Microsoft's "Host ASP.NET Core on Linux with Nginx" documentation: http://mng.bz/yYDp.

So far in this chapter we've used `IHostedService` and the `BackgroundService` to run tasks that repeat on an interval, and you've seen how to install worker services as long-running applications by installing as a Windows Service.

In the final section of this chapter we'll look at how you can create more advanced schedules for your background tasks, as well as how to add resiliency to your application by running multiple instances of your workers. To achieve that, we'll use a mature third-party library, Quartz.NET.

34.3 Coordinating background tasks using Quartz.NET

In this section you'll learn how to use the open-source scheduler library Quartz.NET. You'll learn how to install and configure the library and how to add a background job to run on a schedule. You'll also learn how to enable